Bus-Mapping 개요

목적

Bus-Mapping은 EVM 실행 트레이스(execution trace)를 파싱하고 변환하여 회로 입력 데이터를 생성하는 핵심 모듈입니다. 이 모듈은 Geth와 같은 이더리움 노드에서 수집한 실행 트레이스를 분석하여, ZKEVM 회로가 검증할 수 있는 구조화된 witness 데이터로 변환합니다.

주요 역할

1. **트레이스 파싱**: Geth의 실행 트레이스를 파싱하여 각 opcode 실행 단계를 분석
2. **연산 생성**: 각 실행 단계에서 발생하는 Stack, Memory, Storage 연산을 추출
3. **회로 입력 구축**: 회로가 필요로 하는 witness 데이터를 구조화된 형태로 생성
4. **상태 추적**: 트랜잭션 실행 중 상태 변화를 추적하고 기록

Bus-Mapping의 위치

Bus-Mapping은 EVM 실행 레이어와 회로 레이어 사이의 브릿지 역할을 합니다:

```
EVM 실행 (Geth)

↓

실행 트레이스 (GethExecTrace)

↓

Bus-Mapping (CircuitInputBuilder)

↓
```

```
회로 입력 (Block, Witnesses)

↓

ZKEVM 회로 (SuperCircuit)
```

CircuitInputBuilder 구조

`CircuitInputBuilder` 는 Bus-Mapping의 중심 구조체로, 블록 데이터를 회로 입력으로 변환하는 모든 과정을 관리합니다.

핵심 필드

```rust
pub struct CircuitInputBuilder {
/// StateDB: 계정 상태를 관리하는 key-value 데이터베이스
pub sdb: StateDB,
/// CodeDB: 컨트랙트 코드를 code hash로 관리하는 데이터베이스
pub code_db: CodeDB,
/// Block: 처리 중인 블록 데이터 (실제로는 chunk를 의미)
pub block: Blocks,
/// BlockContext: 블록 처리 중 필요한 컨텍스트 정보
pub block_ctx: BlockContext,
/// MPT 초기 상태 (Scroll의 zkTrie 사용 시)
#[cfg(feature = "scroll")]
pub mpt_init_state: Option<ZktrieState>,
}
```

주요 메서드

1. `handle_block`

블록 전체를 처리하는 최상위 메서드입니다.

```
pub fn handle_block(
&mut self,
eth_block: &EthBlock,
geth_traces: &[GethExecTrace],
) -> Result<(), Error>
```

처리 과정:

1. 블록 내 각 트랜잭션을 순회
2. 각 트랜잭션에 대해 `handle_tx` 호출
3. 모든 트랜잭션 처리 후 `finalize_building` 호출

2. `handle_tx`

개별 트랜잭션을 처리합니다.

```
fn handle_tx(
&mut self,
eth_tx: &eth_types::Transaction,
geth_trace: &GethExecTrace,
) -> Result<(), Error>
```

처리 과정:

1. **BeginTx 단계 생성**: 트랜잭션 시작 시 필요한 연산 생성
2. **Opcode 처리**: 각 `GethExecStep` 에 대해 연산 생성
3. **EndTx 단계 생성**: 트랜잭션 종료 시 필요한 연산 생성

3. `state_ref`

현재 상태에 대한 가변 참조를 제공합니다.

```
pub fn state_ref(
&'a mut self,
tx: &'a mut Transaction,
tx_ctx: &'a mut TransactionContext,
) -> CircuitInputStateRef
```

이 메서드는 opcode 처리 중 상태를 읽고 쓸 수 있는 인터페이스를 제공합니다.

ExecStep과 RwOp의 관계

ExecStep 구조

`ExecStep` 은 EVM 실행의 한 단계를 나타냅니다.

```
pub struct ExecStep {
/// 실행 상태 (Opcode 또는 특수 상태)
pub exec_state: ExecState,
/// 프로그램 카운터
pub pc: ProgramCounter,
```

```rust
/// 스택 크기
pub stack_size: usize,
/// 메모리 크기
pub memory_size: usize,
/// 남은 가스
pub gas_left: Gas,
/// 이 단계의 가스 비용
pub gas_cost: GasCost,
/// 누적 가스 환불
pub gas_refund: Gas,
/// 호출 인덱스
pub call_index: usize,
/// RW 카운터 (이 단계가 실행될 때의 전역 카운터)
pub rwc: RWCounter,
/// 되돌릴 수 있는 쓰기 카운터
pub reversible_write_counter: usize,
/// 이 단계에서 수행된 되돌릴 수 있는 쓰기 연산 수
pub reversible_write_counter_delta: usize,
/// 로그 인덱스
pub log_id: usize,
/// 이 단계에서 생성된 연산들에 대한 참조 목록
pub bus_mapping_instance: Vec<OperationRef>,
/// 복사 이벤트를 통해 수행된 RW 연산 수
```

```
pub copy_rw_counter_delta: u64,

/// 이 단계에서 발생한 에러 (있는 경우)

pub error: Option<ExecError>,

}
```

RwOp (Read-Write Operation)

RwOp는 EVM 실행 중 발생하는 모든 읽기/쓰기 연산을 나타냅니다. Bus-Mapping은 각 `ExecStep`에서 발생하는 모든 RwOp를 생성합니다.

RwOp 타입들

```
pub enum Target {

Start, // 패딩 연산

Memory, // 메모리 연산

Stack, // 스택 연산

Storage, // 영구 스토리지 연산

TransientStorage, // 임시 스토리지 연산 (EIP-1153)

TxAccessListAccount, // 트랜잭션 액세스 리스트 (계정)

TxAccessListAccountStorage, // 트랜잭션 액세스 리스트 (스토리지)

TxRefund, // 트랜잭션 환불

Account, // 계정 상태

CallContext, // 호출 컨텍스트

TxReceipt, // 트랜잭션 영수증

TxLog, // 트랜잭션 로그
```

```
}
```

주요 RwOp 구조체

StackOp: 스택 연산

```
pub struct StackOp {
pub call_id: usize, // 호출 ID
pub address: StackAddress, // 스택 주소
pub value: Word, // 값
}
```

MemoryOp: 메모리 연산

```
pub struct MemoryOp {
pub call_id: usize, // 호출 ID
pub address: MemoryAddress, // 메모리 주소
pub value: Word, // 현재 값
pub value_prev: Word, // 이전 값
}
```

StorageOp: 스토리지 연산

```
pub struct StorageOp {
pub address: Address, // 계정 주소
pub key: Word, // 스토리지 키
```

```
pub value: Word, // 현재 값
pub value_prev: Word, // 이전 값
pub tx_id: usize, // 트랜잭션 ID
pub committed_value: Word, // 트랜잭션 시작 시 값
}
```

ExecStep과 RwOp의 연결

각 `ExecStep` 은 `bus_mapping_instance` 필드를 통해 해당 단계에서 생성된 모든 RwOp에 대한 참조를 보유합니다.

```
ExecStep (SSTORE 실행)
↓
bus_mapping_instance: [
OperationRef → StackOp (key 읽기)
OperationRef → StackOp (value 읽기)
OperationRef → StorageOp (스토리지 쓰기)
OperationRef → TxAccessListAccountStorageOp (액세스 리스트 업데이트)
]
```

데이터 흐름

1. 트레이스 수집

```
Geth 노드
```

```
↓
debug_traceTransaction / debug_traceBlock
↓
GethExecTrace (JSON)
```

2. Bus-Mapping 처리

```
GethExecTrace
↓
CircuitInputBuilder::handle_block
↓
각 트랜잭션 처리
↓
각 GethExecStep 처리
↓
gen_associated_ops (opcode별 연산 생성)
↓
ExecStep + RwOps 생성
```

3. 회로 입력 생성

```
Block (모든 ExecStep 포함)
↓
```

```
OperationContainer (모든 RwOp 정렬)
↓
각 회로별 Witness 생성
↓
회로 할당 및 증명 생성
```

RWCounter의 역할

`RWCounter` 는 모든 읽기/쓰기 연산에 대한 전역 카운터입니다.

특징

- 각 RwOp는 고유한 RWCounter 값을 가짐
- 실행 순서를 보장하는 데 사용
- State Circuit에서 연산의 순서를 검증하는 데 핵심적인 역할

사용 예시

```
// 스택에서 값 읽기
let stack_value = state.stack_pop(&mut exec_step)?;
// 내부적으로 RWCounter가 증가하고 StackOp 생성

// 메모리에 값 쓰기
state.memory_write(&mut exec_step, address, value)?;
```

```
// 내부적으로 RWCounter가 증가하고 MemoryOp 생성
```

CircuitsParams

회로 파라미터는 각 회로가 처리할 수 있는 최대 용량을 정의합니다.

```
pub struct CircuitsParams {
pub max_rws: usize, // State Circuit의 최대 RW 연산 수
pub max_txs: usize, // Tx Circuit의 최대 트랜잭션 수
pub max_calldata: usize, // 최대 calldata 바이트 수
pub max_copy_rows: usize, // Copy Circuit의 최대 행 수
pub max_bytecode: usize, // Bytecode Circuit의 최대 바이트 수
pub max_evm_rows: usize, // EVM Circuit의 최대 행 수
pub max_keccak_rows: usize, // Keccak Circuit의 최대 행 수
// ... 기타 파라미터
}
```

이 파라미터들은 회로 생성 시 결정되며, Bus-Mapping은 이 제약 내에서 데이터를 생성해야 합니다.

에러 처리

Bus-Mapping은 EVM 실행 중 발생하는 다양한 에러를 처리합니다:

- **OutOfGas**: 가스 부족
- **StackOverflow/StackUnderflow**: 스택 오버플로우/언더플로우
- **InvalidOpcode**: 유효하지 않은 opcode

- **InvalidJump**: 유효하지 않은 점프
- **WriteProtection**: 쓰기 보호 위반

이러한 에러는 `ExecStep` 의 `error` 필드에 기록되며, 회로에서 검증됩니다.

관련 문서

- 트레이스 수집: Geth 트레이서 연동 및 트레이스 파싱
- RW 연산: Read-Write 연산 생성 과정
- Opcode 처리: Opcode별 처리 로직
- State Circuit: RW 테이블 검증
- EVM Circuit: Opcode 실행 검증

참고 코드

- `bus-mapping/src/circuit_input_builder.rs` : CircuitInputBuilder 구현
- `bus-mapping/src/operation.rs` : RwOp 타입 정의
- `bus-mapping/src/circuit_input_builder/execution.rs` : ExecStep 정의
- `bus-mapping/src/evm/opcodes/` : Opcode별 처리 로직